From Prompt to State: Verifiable Grounding and Operative Replay for LLM-Mediated Control

Tomas Laurenzo
Department of Critical Media Practices
University of Colorado Boulder
Boulder, Colorado, USA
tomas@laurenzo.net

***Abstract*—When a large language model mediates executable action, an interaction crosses two distinct boundaries: a stochastic model produces text, and an application grounds that text into a deterministic command. Repeating the model call is therefore neither necessary nor generally sufficient to verify the resulting state change. This paper defines an evidence contract for bounded LLM-mediated systems. A trace records the adapter-boundary invocation record, returned text or a commitment to it, the response-to-command grounding, frozen domain rules, semantic events, and pre- and post-action states. Four verification predicates check trace integrity, invocation reconstruction, grounding consistency, and operative replay. We implement the contract in BatLLM, an open-source game in which human instructions are mediated by local language models. The artefact includes a versioned schema, three text-retention profiles, canonical commitments, a provider-neutral recorder, a headless verifier, and an independently implemented reference semantics. In 60 generated sessions comprising 1080 transitions, all recorded states and events replayed equivalently. Full-retention traces reconstructed 360 application-level invocations and independently verified 360 groundings. Differential testing matched production and reference semantics in all 5000 cases. The verifier detected all 1560 applicable re-anchored perturbations and accepted all 180 benign serialisation variants. The result is not model-output reproducibility or generic agent replay; it is a precise, testable account of how retained linguistic evidence produced a domain state.**

***Index Terms*—large language models, provenance, replay, grounding, verification, human–AI interaction**

I. Introduction

Large language models (LLMs) increasingly mediate actions in software environments. A human request may be combined with system instructions and world state, submitted to a model, parsed as a command, and executed by conventional code. Treating this sequence as one operation conceals two different computational regimes. Model inference is probabilistic and can be numerically nondeterministic across hardware, batching, precision, and execution order [1]; post-grounding execution may nevertheless be deterministic.

This distinction matters for reproducibility and audit. Calling a model again does not establish which response was obtained previously, and a prompt–response log does not establish what application state followed. The relevant question for a bounded action system is narrower: can the application reconstruct the invocation it issued, preserve or commit to the response it received, verify the command derived from that response, and replay the command against the same domain semantics?

We investigate this question through BatLLM [4], a turn-based game in which two humans influence software agents through natural-language prompts. Local LLMs return actions in a compact command language; the game then applies movement, rotation, shielding, shooting, collision, and damage rules. This arrangement makes the language-to-action boundary explicit and provides a small but non-trivial deterministic domain.

The paper contributes:

1) an evidence model that distinguishes the application-level invocation, model response, grounding operation, and domain transition;
2) four verification predicates with explicit assumptions and text-retention-dependent availability;
3) a BatLLM implementation comprising a schema, recorder, verifier, canonical commitments, and privacy-aware retention profiles; and
4) an evaluation combining trace replay, independent differential semantics, re-anchored perturbations at multiple trace positions, benign serialisation controls, and overhead measurement.

The claim is deliberately specific. The system does not reproduce a model output, authenticate a trace author, capture transport-layer bytes, or provide general-purpose multi-agent provenance. It verifies the recorded path from an application invocation to a grounded command and from that command to a domain state.

II. Related Work

Games and interactive software have become established environments for evaluating LLM agents. AgentBoard evaluates multi-turn agents through fine-grained progress measures [2]; Cradle addresses general computer control through screen-based interaction [3]. These systems motivate process-level analysis but do not by themselves define a replay contract for an application's language-to-state boundary.

Provenance standards provide a broader foundation. W3C PROV represents entities, activities, agents, and derivations [5]. OpenTelemetry's generative-AI conventions describe ordered input and output messages and model attributes, while explicitly treating captured content as potentially sensitive [6]. Recent

work extends provenance to agentic systems. PROV-AGENT integrates prompts, responses, decisions, and downstream workflow lineage [7]; a recent survey organises evidence tracing and execution provenance around observability, tool use, memory, audit, and recovery [8].

Record-and-replay has also entered LLM research. AgentRR reuses successful trajectories as experience for subsequent tasks [9]. Prompt Scene Investigation records prompt–response evidence for forensic behavioural replay [10]. *agrepl* captures external interactions and replays them without outbound access [11]. Paduraru *et al.* propose Message–Action Traces, executable contracts, replay, and fault injection for agent orchestration [12].

These approaches establish that provenance, trace contracts, and agent replay are not new in themselves. The present contribution concerns a different verification object: the conjunction of application-level invocation reconstruction, response-to-command consistency, and re-execution of an explicit deterministic transition function. It therefore complements rather than replaces transport replay, workflow provenance, behavioural re-enactment, or orchestration assurance.

III. Evidence and Verification Contract

A. Execution model

For step i, let u_i be the human instruction, h_i the ordered conversational history, s_i the pre-action state, θ_i the declared model configuration, and ρ_i the frozen domain rules. The application constructs an adapter-boundary invocation record

$$a_i = A(u_i, h_i, s_i, \theta_i), \tag{1}$$

then obtains response text y_i from model provider M. A parser P grounds the response into command c_i, and a transition function δ produces the next state and ordered semantic events:

$$c_i = P(y_i), \qquad (s_{i+1}, e_i) = \delta(s_i, c_i; \rho_i). \tag{2}$$

The trace unit is

$$T_i = \langle u_i, h_i, s_i, a_i, \theta_i, y_i, c_i, \rho_i, e_i, s_{i+1} \rangle. \tag{3}$$

Here a_i is the record materialised immediately before the provider-adapter call. Its model, ordered messages, options, and stream fields correspond to call arguments; provider and endpoint are application declarations. It is not the provider library's internal representation or the HTTP request emitted below that boundary.

B. Verification predicates

The contract defines four predicates rather than a single undifferentiated notion of reproducibility.

V1—Trace validity. The schema, identifiers, sequence, state continuity, final states, content commitments, transition commitments, and ordered play chain are internally consistent.

V2—Invocation evidence. In full retention, the verifier independently reconstructs the current user message and conversation history from separately recorded prompt, state, system instructions, and context policy, then compares the result with a_i. Provider, endpoint, and requested model are checked against session provenance. Reduced-retention profiles provide weaker evidence, as Table I specifies.

TABLE I
EVIDENCE AVAILABLE BY TEXT-RETENTION PROFILE. "COMMIT." DENOTES A SHA-256 COMMITMENT WITHOUT RETAINED TEXT.

Predicate	Full	Redacted	Hash-only
V1 trace	yes	yes	yes
V2 invocation	reconstruct	struct.+hash	hash
V3 grounding	verify	no	no
V4 replay	verify	verify	verify

V3—Grounding consistency. When y_i is retained, re-applying P must yield the recorded c_i and validity status. Provider failures are represented separately and resolve operatively to `ERR`; they are not counted as model-response groundings.

V4—Operative replay. Re-evaluating $\delta(s_i, c_i; \rho_i)$ must reproduce both s_{i+1} and e_i. BatLLM compares discrete state exactly and position and rotation with absolute tolerance 10^{-6}.

Internal semantic soundness. Assume that P and δ are deterministic and that the verifier executes the semantics identified by the trace's application revision. If V3 and V4 hold for a full-retention step, then the recorded command equals $P(y_i)$ and the recorded domain-state projection and events equal $\delta(s_i, P(y_i); \rho_i)$. This implication concerns internal semantic consistency. Inferring that the trace is a historically authentic record of an earlier execution additionally requires a trusted recorder or an independently anchored commitment. It does not imply that another invocation of M would regenerate y_i.

C. Threat and disclosure boundaries

Canonical SHA-256 values expose accidental corruption and partial edits by making stale commitments observable. An independently preserved commitment is required to detect a comprehensive rewrite in which every internal digest is recomputed. These values are unkeyed commitments, not signatures: a party able to replace the complete trace and all external anchors can construct another internally valid record. Model names and options are declarations; model weights, provider defaults, and serving binaries are not authenticated.

The three profiles govern prompt, system-instruction, response, and request-message text, not all session metadata or game state. Hash-only storage is not confidentiality: low-entropy prompts may be recoverable by guessing and hashing. Redaction also prevents independent V3 verification. These constraints are properties of the evidence retained, not incidental limitations of the verifier.

IV. Implementation

BatLLM's normalised command forms are movement (`M`, `Md`), clockwise and counter-clockwise rotation (`C`α, `A`α), shooting (`B`), and shield operations (`S`, `S1`, `S0`); unsupported text in the evaluation resolves to `ERR`. Gameplay is implemented as a pure transition module independent of the Kivy interface. The

V4 equivalence relation projects each bot onto health, position, rotation, and shield state; prompt and diagnostic fields belong to invocation evidence rather than gameplay state.

The v3 research runtime is a headless execution path that uses the same production parser and transition engine. Immediately before adapter invocation, it materialises the ordered messages, requested provider, endpoint declaration, model, generation options, and stream setting. It records timestamps, latency, retry count, response or error, normalised command, rules, states, and semantic events. Failed attempts are removed from subsequent conversation history, matching the application's rollback policy. The existing graphical application's v2 user export remains unchanged; this paper's claims concern the explicit v3 research path.

Canonical JSON uses UTF-8, ordered keys, compact separators, and rejects non-finite numbers. Separate commitments cover protected text, retained requests, transitions, play records, the ordered play chain, and the session envelope. Semantic events exclude raw model text, which remains only in the retention-governed invocation evidence. The verifier reports predicate-specific failures in text or JSON and exits non-zero. The source, schema, experiments, reviewer ledger, and build instructions are distributed in the BatLLM repository's `urucon` branch.

To reduce dependence on a shared implementation, the artefact includes a second executable semantics that imports no production gameplay module. It separately implements command normalisation, movement bounds, rotation, shields, projectile traversal, collision, damage, state normalisation, and semantic-event construction. This is not a formal proof and can share specification errors, but it converts replay from a same-function regression into differential testing between two implementations.

V. Evaluation

A. Questions and protocol

We evaluate four questions. *Q1*: do generated traces satisfy the evidence available under each retention profile and reproduce their recorded state and events? *Q2*: does the production transition engine agree with the independent executable semantics? *Q3*: does the verifier reject controlled semantic inconsistencies while accepting semantically identical JSON serialisations? *Q4*: what canonical storage cost does the reference representation impose?

The trace corpus contains 60 sessions and 1080 transitions: twenty sessions per retention profile, eighteen transitions per session, four requested model identifiers, shared and independent conversation histories, augmented and unaugmented prompts, varied gameplay parameters, every documented command form, and invalid text. Responses are scripted. This isolates trace semantics from model competence and network availability; it is not an evaluation of LLM behaviour. Incidental identifiers, timestamps, and latency fields are stabilised, while environment and source-revision provenance are retained. All 60 traces validate against the published JSON Schema Draft 2020-12 with date-time format checking.

TABLE II
Reference artefact results.

Measure	Result
Schema-valid / semantically valid	60 / 60
State / event replays	1080 / 1080
Full invocation reconstructions	360
Retained-text groundings verified	360
Differential state/event/path cases	5000/5000/5000
Re-anchored perturbations rejected	1560/1560
Benign serialisations accepted	180/180
Raw bytes/play: full/redacted/hash	4374/4741/2203
Mean gzip bytes/play	439

Differential testing uses a fixed seed and 5000 targeted and random cases. Cases vary positions, rotation, health, shields, rules, commands, invalid text, and projectile outcomes. Production and reference implementations are compared for normalisation, resulting state, ordered events, and projectile path.

The perturbation experiment changes eleven classes of evidence: command, retained response, retained invocation, retained human prompt, state supplied to the model, rules, pre-state, post-state, events, deletion, and reordering. Play-local faults are applied in early, middle, and late positions; rule faults are session-level. Mutations unavailable under a retention profile are excluded. Before verification, all commitments derivable from retained mutated content are recomputed. A second control reserialises every valid trace using compact ASCII, pretty Unicode, and recursively reversed key order.

Storage is measured on canonical UTF-8 JSON and gzip level 9. The artefact also records repeated verifier timing as diagnostic data, but the paper does not treat shared-runner timing as a portable performance result.

B. Results

All sessions passed both the published schema and the semantic guarantees available under their retention profile (Table II). Every recorded domain state and event list replayed equivalently. Full retention reconstructed 360 adapter-boundary invocation records and verified 360 response-to-command mappings. The 720 reduced-retention transitions remained operatively replayable without being misreported as independently grounded.

Production and reference semantics agreed in all 5000 cases for command normalisation, state, events, and projectile paths. The verifier rejected all 1560 applicable re-anchored perturbations across trace positions and accepted all 180 benign variants, yielding no observed false rejection in this control set. These are exhaustive results only for the declared fixtures, not estimates of attack detection in deployment.

Raw storage varied substantially by retention profile. Redacted traces were slightly larger than full traces because each removed message content was replaced by a structured commitment; hash-only traces were smallest. Compression reduced the overall mean to 439 bytes per play.

VI. Discussion

The contract separates three questions that are often collapsed in agent logs. First, what application-level evidence exists for the model interaction? Second, is the recorded linguistic response consistent with the command attributed to it? Third, does that command entail the recorded domain consequence? A full-retention trace can answer all three. Reduced-retention traces deliberately trade the second answer for lower textual disclosure while preserving operative replay. The distinction prevents a replayable command log from being misrepresented as evidence of what a model said.

The evaluation also distinguishes internal consistency from historical authenticity. Re-anchored perturbations show that cross-field and semantic constraints detect more than stale hashes; benign serialisations show that this strictness is not merely byte-level canonicality. Nevertheless, a comprehensively rewritten trace can remain internally valid. External anchoring is therefore orthogonal to semantic verification.

BatLLM is one bounded instance, not empirical evidence of cross-domain generality. The transferable design condition is explicit: model text must cross a visible grounding boundary into a command language whose transition semantics can be frozen and re-executed. Tool-calling interfaces, workflow languages, and constrained robotic command layers may satisfy this condition, but each requires its own state projection, event vocabulary, and independent semantic tests.

VII. Limitations and Conclusion

The implementation validates a bounded deterministic domain. Systems with stochastic post-grounding execution must additionally capture random state or external effects. The independent semantics reduces, but cannot eliminate, common-specification error. The corpus uses scripted responses and does not establish live-provider completeness, model quality, or human usefulness. The recorded invocation is application-level rather than transport-level, and declared model identity is not cryptographically tied to weights or serving code. Retention commitments do not provide confidentiality or authorship, and reduced retention sacrifices independent grounding verification. Finally, the graphical application does not yet emit v3 traces; compatibility is preserved through a separate research runtime.

Within these boundaries, the contract establishes a useful separation: model generation need not be repeatable for the action derived from a recorded response to be verifiable. By retaining evidence at the adapter boundary, exposing grounding as a checkable operation, and replaying explicit domain semantics, BatLLM provides a concrete account of how human instruction became computational state. The pattern applies to other bounded LLM-mediated systems whose outputs are grounded into an explicit command language and deterministic transition function.

Acknowledgement

BatLLM was supported by the University of Colorado Boulder's Center for Humanities & the Arts and Research & Innovation Office.

References

[1] J. Yuan *et al.*, "Understanding and mitigating numerical sources of nondeterminism in LLM inference," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[2] C. Ma *et al.*, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[3] W. Tan *et al.*, "Cradle: Empowering foundation agents towards general computer control," in *Proc. 42nd Int. Conf. Machine Learning*, 2025, pp. 58658–58725.

[4] T. Laurenzo, "BatLLM: Local-first human–LLM-mediated play and analysis," open-source software, 2026.

[5] W3C Provenance Working Group, "PROV-O: The PROV ontology," W3C Recommendation, 2013.

[6] OpenTelemetry Authors, "Generative AI semantic conventions: Attribute registry," OpenTelemetry Specification, 2026.

[7] R. Souza *et al.*, "PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows," in *Proc. IEEE Int. Conf. eScience*, 2025, pp. 467–473.

[8] Y. Wang *et al.*, "From agent traces to trust: Evidence tracing and execution provenance in LLM agents," arXiv:2606.04990, 2026.

[9] E. Feng *et al.*, "Get experience from practice: LLM agents with record & replay," arXiv:2505.17716, 2025.

[10] L. F. Ledjaki, Y.-C. Chang, M. Loutis, and E. Aïmeur, "Prompt scene investigation: Uncovering the evidence in LLMs," in *Companion Proc. ACM Web Conf.*, 2026, pp. 284–290.

[11] R. Mudasiru, "Deterministic replay for AI agent systems," arXiv:2607.16200, 2026.

[12] C. Paduraru, P.-L. Bouruc, and A. Stefanescu, "A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance," in *Proc. 21st Int. Conf. Evaluation of Novel Approaches to Software Engineering*, 2026, pp. 420–427.